```
import os
import re
import fitz  # PyMuPDF
import spacy
from flask import Flask, request
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.metrics.pairwise import cosine_similarity

import matplotlib.pyplot as plt
import io
import base64

# --- 1. SETUP ---
app = Flask(__name__)
UPLOAD_FOLDER = 'uploads'
if not os.path.exists(UPLOAD_FOLDER):
    os.makedirs(UPLOAD_FOLDER)
app.config['UPLOAD_FOLDER'] = UPLOAD_FOLDER

try:
    nlp = spacy.load("en_core_web_sm")
except OSError:
    print("Downloading spaCy model 'en_core_web_sm'...")
    from spacy.cli import download
    download("en_core_web_sm")
    nlp = spacy.load("en_core_web_sm")

# --- 2. SKILLS_DB & HELPER FUNCTIONS ---
SKILLS_DB = [
    # --- Programming Languages ---
    'python', 'java', 'c++', 'c#', 'javascript', 'typescript', 'php', 'ruby', 'swift', 'kotlin',
    'go', 'rust', 'sql', 'r', 'matlab', 'powershell', 'bash','c', 'perl', 'scala', 'haskell','mysql'

    # --- Web Development & Frameworks ---
    'html', 'css', 'react', 'angular', 'vue', 'next.js', 'node.js', 'express.js', 'django',
    'flask', 'ruby on rails', 'asp.net', 'spring boot', 'jquery', 'bootstrap',

    # --- Databases & Data Storage ---
    'mysql', 'postgresql', 'sqlite', 'mongodb', 'redis', 'cassandra', 'microsoft sql server',
    'oracle', 'nosql', 'firebase', 'dynamodb',

    # --- Cloud & DevOps ---
    'aws', 'azure', 'google cloud', 'gcp', 'docker', 'kubernetes', 'terraform', 'ansible',
    'jenkins', 'git', 'github actions', 'circleci', 'vagrant', 'serverless', 'ci/cd',

    # --- Data Science & Machine Learning ---
    'machine learning', 'deep learning', 'data analysis', 'data visualization', 'statistics',
    'natural language processing', 'nlp', 'computer vision', 'tensorflow', 'pytorch',
    'scikit-learn', 'keras', 'pandas', 'numpy', 'scipy', 'matplotlib', 'seaborn',
    'apache spark', 'hadoop',

    # --- Business Intelligence & Data Warehousing ---
    'tableau', 'power bi', 'qlik sense', 'looker', 'etl', 'data warehousing',
    'business intelligence',

    # --- Mechanical & Industrial Engineering ---
    'solidworks', 'autocad', 'catia', 'creo', 'proe', 'revit', 'siemens nx', 'ansys',
    'simulink', 'abaqus', 'labview', 'fea', 'finite element analysis', 'cfd',
    'computational fluid dynamics', 'gdt', 'geometric dimensioning and tolerancing',

    'dfm', 'dfa', 'hvac', 'mep', 'lean manufacturing', 'six sigma', 'bom', 'bill of materials',
    'plm', 'product lifecycle management', 'cnc machining', 'injection molding', '3d printing',
    'robotics', 'mechatronics',

    # --- Project & Product Management ---
    'agile', 'scrum', 'kanban', 'waterfall', 'prince2', 'pmp', 'jira', 'confluence',
    'asana', 'trello', 'ms project', 'product roadmap', 'user stories', 'sprint planning',

    # --- IT & Networking ---
    'tcp/ip', 'dns', 'dhcp', 'vpn', 'firewalls', 'load balancing', 'cybersecurity',
    'cisco', 'juniper', 'active directory', 'wireshark', 'ccna', 'cissp', 'comptia security+',

    # --- Soft Skills (for basic matching) ---
    'leadership', 'communication', 'teamwork', 'problem-solving', 'collaboration',
    'creativity', 'mentorship', 'project management', 'analysis'
]
```

```
]


def extract_text_from_pdf(pdf_path):
    try:
        doc = fitz.open(pdf_path)
        text = ""
        for page in doc:
            text += page.get_text()
        return text
    except Exception as e:
        print(f"Error extracting text from PDF {pdf_path}: {e}")
        return ""

def preprocess_text(text):
    doc = nlp(text.lower())
    tokens = [token.lemma_ for token in doc if not token.is_stop and not token.is_punct and not token.is_space]
    return " ".join(tokens)

def calculate_similarity_score(resume_text, job_description_text):
    text_corpus = [resume_text, job_description_text]
    vectorizer = TfidfVectorizer()
    try:
        tfidf_matrix = vectorizer.fit_transform(text_corpus)
        return cosine_similarity(tfidf_matrix)[0, 1]
    except ValueError:
        return 0.0

# --- 3. HTML TEMPLATES ---
HTML_INDEX_PAGE = f"""
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>AI Resume Matcher</title>
    <link href="https://fonts.googleapis.com/css2?family=Inter:wght@300;400;600;700&display=swap" rel="stylesheet">
    <style>
        :root {{
            --primary-color: #0056b3; /* Darker blue */
            --secondary-color: #f0f2f5; /* Light grey background */
            --text-color: #333;
            --light-text-color: #666;
            --border-color: #e0e0e0;
            --success-color: #28a745;
            --warning-color: #ffc107;
            --danger-color: #dc3545;
        }}
        body {{
            font-family: 'Inter', sans-serif;
            background-color: var(--secondary-color);
            margin: 0;
            padding: 0;
            color: var(--text-color);
            line-height: 1.6;
        }}
        .header {{
            background-color: var(--primary-color);
            color: white;
            padding: 15px 0;
            text-align: center;
            box-shadow: 0 2px 4px rgba(0,0,0,0.1);
        }}
        .header h1 {{
            margin: 0;
            font-weight: 600;
        }}
        .container {{
            max-width: 900px;
            margin: 40px auto;
            background: #fff;
            padding: 30px;
            border-radius: 10px;
            box-shadow: 0 4px 15px rgba(0,0,0,0.1);
            animation: fadeIn 0.8s ease-out;
        }}
        @keyframes fadeIn {{
            from {{ opacity: 0; transform: translateY(20px); }}
```

```
            to {{ opacity: 1; transform: translateY(0); }}
        }}
        h2 {{
            color: var(--primary-color);
            text-align: center;
            margin-bottom: 30px;
            font-weight: 700;
        }}
        .form-group {{
            margin-bottom: 25px;
        }}
        label {{
            display: block;
            margin-bottom: 8px;
            font-weight: 600;
            color: var(--light-text-color);
            font-size: 0.95em;
        }}
        textarea, input[type="file"] {{
            width: calc(100% - 22px); /* Account for padding and border */
            padding: 12px;
            border-radius: 6px;
            border: 1px solid var(--border-color);
            box-sizing: border-box;
            font-family: 'Inter', sans-serif;
            font-size: 1em;
            transition: border-color 0.2s;
        }}
        textarea:focus, input[type="file"]:focus {{
            border-color: var(--primary-color);
            outline: none;
        }}
        input[type="file"] {{
            padding: 10px 12px; /* Adjust for better file input appearance */
        }}
        button {{
            display: block;
            width: 100%;
            padding: 14px;
            background-color: var(--primary-color);
            color: white;
            border: none;
            border-radius: 6px;
            font-size: 1.1em;
            font-weight: 600;
            cursor: pointer;
            transition: background-color 0.2s ease, transform 0.1s ease;
        }}
        button:hover {{
            background-color: #004494; /* Slightly darker primary */
            transform: translateY(-1px);
        }}
        button:active {{
            transform: translateY(0);
        }}
        .note {{
            text-align: center;
            margin-top: 20px;
            color: var(--light-text-color);
            font-size: 0.85em;
        }}
    </style>
</head>
<body>
    <div class="header">
        <h1>Resume Analyser and Ranking System</h1>
    </div>
    <div class="container">
        <h2>Rank Resumes Against a Job Description</h2>
        <p class="note">This tool combines overall text similarity with specific skill matching to provide a comprehensive rankin
        <form method="post" enctype="multipart/form-data">
            <div class="form-group">
                <label for="job_description">Job Description</label>
                <textarea name="job_description" id="job_description" rows="12" required placeholder="Paste the full job descrip
            </div>
            <div class="form-group">
                <label for="resumes">Upload Resumes (PDFs)</label>
```

```
                <input type="file" name="resumes" id="resumes" multiple required accept=".pdf">
            </div>
            <button type="submit">Analyze & Rank Resumes</button>
        </form>
    </div>
</body>
</html>
"""

# --- Plotting Functions ---
def generate_plots_for_resume(result):
    # Stacked Bar Chart (Matched vs Missing Skills)
    skill_match_breakdown_plot = ""
    if result['skills_required_count'] > 0:
        matched_percent = (result['skills_matched_count'] / result['skills_required_count']) * 100
        missing_percent = ((result['skills_required_count'] - result['skills_matched_count']) / result['skills_required_count'])

        fig, ax = plt.subplots(figsize=(6, 1.5))
        ax.barh(0, matched_percent, color='#28a745', label='Matched Skills') # Green
        ax.barh(0, missing_percent, left=matched_percent, color='#dc3545', label='Missing Skills') # Red

        ax.set_xlim(0, 100)
        ax.set_yticks([])
        ax.set_xticks([0, 25, 50, 75, 100])
        ax.set_xlabel('Percentage of Required Skills')
        ax.set_title('Skill Match Breakdown', fontsize=10)
        ax.legend(loc='center left', bbox_to_anchor=(1, 0.5), fontsize=8)
        ax.spines['right'].set_visible(False)
        ax.spines['left'].set_visible(False)
        ax.spines['top'].set_visible(False)
        plt.tight_layout()

        buf = io.BytesIO()
        plt.savefig(buf, format='png', bbox_inches='tight')
        plt.close(fig)
        skill_match_breakdown_plot = base64.b64encode(buf.getvalue()).decode('utf-8')

    return skill_match_breakdown_plot


def generate_overall_scatter_plot(all_results):
    overall_scatter_plot_img = ""
    if all_results:
        fig, ax = plt.subplots(figsize=(10, 7)) # Adjust figure size as needed

        similarity_scores = [r['similarity_score'] / 100 for r in all_results]
        skill_scores = [r['skill_score'] / 100 for r in all_results]
        final_scores = [r['final_score'] for r in all_results]
        filenames = [r['filename'] for r in all_results]

        sizes = [max(200, fs * 10) for fs in final_scores] # Ensure a minimum visible size

        scatter = ax.scatter(similarity_scores, skill_scores, s=sizes, alpha=0.6, c='#007bff', edgecolors='w', linewidth=0.5)

        for i, txt in enumerate(filenames):
             ax.annotate(txt, (similarity_scores[i], skill_scores[i]), textcoords="offset points", xytext=(0,10), ha='center', f

        ax.set_title('Overall Skill vs. Similarity Plot (All Resumes)', fontsize=14)
        ax.set_xlabel('Text Similarity Score (0-1)')
        ax.set_ylabel('Skill Match Score (0-1)')
        ax.set_xlim(-0.05, 1.05)
        ax.set_ylim(-0.05, 1.05)
        ax.grid(True, linestyle='--', alpha=0.7)

        plt.tight_layout()
        buf = io.BytesIO()
        plt.savefig(buf, format='png', bbox_inches='tight')
        plt.close(fig)
        overall_scatter_plot_img = base64.b64encode(buf.getvalue()).decode('utf-8')

    return overall_scatter_plot_img


# ***** UPDATED: generate_results_page for professional UI/UX and integrated recommendations + Plots *****
def generate_results_page(ranked_results):
    table_rows = ""

    for idx, result in enumerate(ranked_results):
```

```
    for idx, result in enumerate(ranked_results):
        rank = idx + 1

        # Determine score class for visual feedback
        score_class = 'score-high'
        if result['final_score'] < 50:
            score_class = 'score-low'
        elif result['final_score'] < 75:
            score_class = 'score-medium'

        # Format the matched skills for display
        matched_skills_str = ", ".join(sorted(list(result['matched_skills_set'])))
        if not matched_skills_str:
            matched_skills_str = "<span class='text-muted'>None identified</span>"

        # Format recommendations
        recommendations = []
        if result['skills_required_count'] == 0:
            recommendations.append("<p class='text-muted'>No specific skills from our database were found in the Job Description
        else:
            if result['missing_skills_set']:
                missing_str = ", ".join(sorted(list(result['missing_skills_set'])))
                recommendations.append(f"To improve, highlight experience with: <strong>{missing_str}</strong>.")
            # Only add the "Excellent match" if there are required skills and no missing skills
            elif result['skills_required_count'] > 0 and not result['missing_skills_set']:
                recommendations.append("<span class='text-success'>Excellent match for identified skills!</span>")
            # If there are required skills, but none were matched
            elif result['skills_matched_count'] == 0 and result['skills_required_count'] > 0:
                 recommendations.append("<span class='text-danger'>No identified job skills from the JD were found in this resume


        recommendations_str = "<br>".join(recommendations) if recommendations else "No specific recommendations."

        # Generate the individual skill breakdown plot for the current resume
        skill_breakdown_plot_img = generate_plots_for_resume(result)

        table_rows += f"""
        <tr>
            <td>{rank}</td>
            <td>{result['filename']}</td>
            <td class='{score_class}'><strong>{result['final_score']:.2f}%</strong></td>
            <td>{result['similarity_score']:.2f}%</td>
            <td>{result['skill_score']:.2f}% ({result['skills_matched_count']} / {result['skills_required_count']})</td>
            <td>{matched_skills_str}</td>
            <td>{recommendations_str}
                <details>
                    <summary style="padding: 5px 0;">View Visual Breakdown</summary>
                    <div class="plot-container">
                        <h3>Breakdown for {result['filename']}</h3>
                        {'<img src="' + skill_breakdown_plot_img + '" alt="Skill Match Breakdown" />' if sk
                    </div>
                </details>
            </td>
        </tr>
        """

    # Generate the overall scatter plot outside the loop
    overall_scatter_plot_img = generate_overall_scatter_plot(ranked_results)

    return f"""
    <!DOCTYPE html>
    <html lang="en">
    <head>
        <meta charset="UTF-8">
        <meta name="viewport" content="width=device-width, initial-scale=1.0">
        <title>Ranking Results - AI Resume Matcher</title>
        <link href="https://fonts.googleapis.com/css2?family=Inter:wght@300;400;600;700&display=swap" rel="stylesheet">
        <style>
            :root {{
                --primary-color: #0056b3; /* Darker blue */
                --secondary-color: #f0f2f5; /* Light grey background */
                --text-color: #333;
                --light-text-color: #666;
                --border-color: #e0e0e0;
                --header-bg-color: #f8f9fa; /* Lighter header for tables */
                --score-high: #28a745; /* Green */
                --score-medium: #ffc107; /* Yellow/Orange */
                --score-low: #dc3545; /* Red */
```

```
        }}
        body {{
            font-family: 'Inter', sans-serif;
            background-color: var(--secondary-color);
            margin: 0;
            padding: 0;
            color: var(--text-color);
            line-height: 1.6;
        }}
        .header {{
            background-color: var(--primary-color);
            color: white;
            padding: 15px 0;
            text-align: center;
            box-shadow: 0 2px 4px rgba(0,0,0,0.1);
        }}
        .header h1 {{
            margin: 0;
            font-weight: 600;
        }}
        .container {{
            max-width: 1400px; /* Increased max-width for results table */
            margin: 40px auto;
            background: #fff;
            padding: 30px;
            border-radius: 10px;
            box-shadow: 0 4px 15px rgba(0,0,0,0.1);
            animation: fadeIn 0.8s ease-out;
        }}
        @keyframes fadeIn {{
            from {{ opacity: 0; transform: translateY(20px); }}
            to {{ opacity: 1; transform: translateY(0); }}
        }}
        h2 {{
            color: var(--primary-color);
            text-align: center;
            margin-bottom: 30px;
            font-weight: 700;
        }}
        table {{
            width: 100%;
            border-collapse: separate; /* Use separate to allow border-radius on cells */
            border-spacing: 0;
            margin-top: 20px;
            font-size: 0.9em;
        }}
        th, td {{
            padding: 15px 12px;
            border-bottom: 1px solid var(--border-color);
            text-align: left;
            vertical-align: top;
        }}
        th {{
            background-color: var(--header-bg-color);
            color: var(--primary-color);
            font-weight: 700;
            text-transform: uppercase;
            font-size: 0.85em;
            border-top: 1px solid var(--border-color); /* Add top border for aesthetic */
        }}
        th:first-child {{ border-top-left-radius: 8px; }}
        th:last-child {{ border-top-right-radius: 8px; }}

        tr:last-child td {{
            border-bottom: none;
        }}
        tr:hover {{
            background-color: #f5f5f5;
        }}
        .score-high {{ color: var(--score-high); font-weight: 600; }}
        .score-medium {{ color: var(--score-medium); font-weight: 600; }}
        .score-low {{ color: var(--score-low); font-weight: 600; }}
        .text-success {{ color: var(--score-high); }}
        .text-danger {{ color: var(--score-low); }}
        .text-muted {{ color: var(--light-text-color); font-style: italic; }}

        .back-link {{
            display: inline-block;
```

```
                display: inline-block;
                margin-top: 30px;
                text-decoration: none;
                padding: 12px 25px;
                background-color: var(--primary-color);
                color: white;
                border-radius: 6px;
                font-size: 1em;
                font-weight: 600;
                transition: background-color 0.2s ease, transform 0.1s ease;
            }}
            .back-link:hover {{
                background-color: #004494;
                transform: translateY(-1px);
            }}
            .back-link-container {{
                text-align: center;
                margin-top: 30px; /* Add margin to separate from plot */
            }}

            /* Styles for Plot Containers */
            .plot-section {{
                background-color: #fdfdfd;
                border-radius: 8px;
                padding: 25px;
                margin-bottom: 30px;
                box-shadow: 0 2px 10px rgba(0,0,0,0.05);
                text-align: center;
            }}
            .plot-section h3 {{
                color: var(--primary-color);
                margin-top: 0;
                margin-bottom: 20px;
                font-weight: 600;
            }}
            .plot-container img {{
                max-width: 100%;
                height: auto;
                border: 1px solid var(--border-color);
                border-radius: 5px;
                margin: 10px 0;
            }}

            /* Styles for Details/Summary (accordion for plots) */
            details {{
                margin-top: 10px;
                background-color: #e9f7ff; /* Light blue background for details */
                border-radius: 5px;
                padding: 5px 10px;
                border: 1px solid #cce5ff;
            }}
            summary {{
                font-weight: 600;
                color: var(--primary-color);
                cursor: pointer;
                padding: 5px 0; /* Added padding here */
                outline: none; /* Remove default focus outline */
            }}
            summary:hover {{
                color: #004494;
            }}
            details[open] summary {{
                border-bottom: 1px solid #cce5ff;
                margin-bottom: 10px;
            }}
            .plot-container {{
                padding: 15px 0 5px 0;
            }}
        </style>
    </head>
    <body>
        <div class="header">
            <h1>Resume Analyser and Ranking System</h1>
        </div>
        <div class="container">
            <h2>Resume Ranking Results</h2>

            <table>
```

```
                    <thead>
                        <tr>
                            <th>Rank</th>
                            <th>Resume File</th>
                            <th>Final Score</th>
                            <th>Text Similarity</th>
                            <th>Skill Match (Found/Req.)</th>
                            <th>Matched Skills</th>
                            <th>Recommendations & Details</th>
                        </tr>
                    </thead>
                    <tbody>
                        {table_rows}
                    </tbody>
                </table>

                <div class="plot-section">
                    <h3>Overall Skill vs. Similarity Overview</h3>
                    <p class="text-muted">Each bubble represents a resume. Size indicates final score.</p>
                    {'<img src="' + overall_scatter_plot_img + '" alt="Overall Skill vs Similarity Plot" />' i
                </div>

                <div class="back-link-container">
                    <a href="/" class="back-link">Analyze More Resumes</a>
                </div>
            </div>
        </body>
        </html>
        """



# --- 4. FLASK ROUTING ---
@app.route('/', methods=['GET', 'POST'])
def index():
    if request.method == 'POST':
        job_description = request.form['job_description']
        uploaded_files = request.files.getlist('resumes')

        results = []

        # Preprocess JD once
        # Find required skills from JD once
        jd_skills_required = {skill for skill in SKILLS_DB if re.search(r'\b' + re.escape(skill) + r'\b', job_description.lower(

        # Only preprocess JD text for similarity score if needed.
        # For skill matching, we use the raw JD text.
        clean_jd_for_similarity = preprocess_text(job_description)

        for file in uploaded_files:
            if file.filename != '' and file.filename.lower().endswith('.pdf'):
                filepath = os.path.join(app.config['UPLOAD_FOLDER'], file.filename)
                file.save(filepath)

                raw_resume_text = extract_text_from_pdf(filepath)
                clean_resume_for_similarity = preprocess_text(raw_resume_text)

                similarity_score = calculate_similarity_score(clean_resume_for_similarity, clean_jd_for_similarity)

                skills_matched = {skill for skill in jd_skills_required if re.search(r'\b' + re.escape(skill) + r'\b', raw_resume

                skill_score = 0.0
                if jd_skills_required:
                    skill_score = len(skills_matched) / len(jd_skills_required)

                missing_skills = jd_skills_required - skills_matched

                similarity_weight = 0.4
                skill_weight = 0.6
                final_score = (similarity_weight * similarity_score + skill_weight * skill_score) * 100

                results.append({
                    'filename': file.filename,
                    'final_score': final_score,
                    'similarity_score': similarity_score * 100,
                    'skill_score': skill_score * 100,
                    'skills_matched_count': len(skills_matched),
```

```
                        'skills_required_count': len(jd_skills_required),
                        'matched_skills_set': skills_matched,
                        'missing_skills_set': missing_skills
                    })

                    os.remove(filepath)

            ranked_results = sorted(results, key=lambda x: x['final_score'], reverse=True)
            return generate_results_page(ranked_results)

        return HTML_INDEX_PAGE

    # --- 5. RUN THE APP ---
    if __name__ == '__main__':
        app.run(debug=True)
```